6.1220: Design and Analysis of Algorithms

Ace Chun

July 6, 2026

Contents

1 Analysis Techniques **3**
- 1.1 Amortized Analysis . . . 3
  - 1.1.1 Aggregate Method . . . 3
  - 1.1.2 Potential Method . . . 3
- 1.2 Competitive Analysis . . . 4
- 1.3 Probabilistic Analysis . . . 5

2 Data Structures **6**
- 2.1 Union-Find . . . 6
- 2.2 Hash Tables . . . 7
  - 2.2.1 Chaining . . . 8
  - 2.2.2 Open Addressing . . . 9
  - 2.2.3 Perfect Hashing . . . 10

3 Deterministic Polynomial-Time Algorithms **11**
- 3.1 Greedy . . . 11
  - 3.1.1 Interval Scheduling . . . 12
  - 3.1.2 Minimum Spanning Trees . . . 14
- 3.2 Dynamic Programming . . . 16
  - 3.2.1 Weighted Interval Scheduling . . . 16
- 3.3 Divide and Conquer . . . 18
  - 3.3.1 Rank-Finding . . . 18
  - 3.3.2 Fast Fourier Transform . . . 20
- 3.4 Max-Flow and Min-Cut . . . 20
  - 3.4.1 Duality . . . 21

3.4.2 Max-Flow . . . 22
3.4.3 Min-Cut . . . 23

4 Randomization **23**
4.1 Selecting a pivot . . . 24
4.2 Markov Chains and Random Walks . . . 24
4.2.1 Convergence . . . 25
4.2.2 Connectivity . . . 25
4.2.3 Reversibility . . . 26
4.2.4 Fundamental Theorem of Markov Chains . . . 26
4.2.5 Metropolis-Hastings . . . 26

5 Online Algorithms **27**
5.1 Online Interval Scheduling . . . 28
5.2 Self Organizing Lists . . . 29
5.3 Online Learning . . . 31
5.3.1 Weighted Majority . . . 32
5.3.2 Multiplicative Weights . . . 34

6 Linear Programming **34**
6.1 Duality . . . 36
6.2 Complementary Slackness . . . 37

7 Approximation **37**
7.1 Greedy Approximation . . . 38
7.2 LP Rounding . . . 39
7.3 Randomized Approximations . . . 39
7.3.1 Derandomization . . . 39
7.4 Sublinear Optimization . . . 40
7.5 Property Testing . . . 40

8 Streaming Algorithms **40**
8.1 Sketching . . . 41
8.1.1 Min-Hashing . . . 41
8.1.2 Majority Counters . . . 41
8.2 Similarity Search . . . 42

1 Analysis Techniques

1.1 Amortized Analysis

Amortized analysis is interested in the amount of time needed to achieve a *series* of computations (typically with respect to a data structure), instead of analyzing the worst case complexity of each computation in isolation. This comes from the recognition that, often, the more expensive operations in a given data structure must necessarily be preceded by several cheaper operations (e.g. resizing a dynamic array must be preceded by a certain number of insertions). An amortized time bound takes the average cost over this sequence, "distributing" the expensive operation over several cheap ones.

Formally, a data structure D has an amortized cost of T if any sequence of k operations on D starting from some initial state has a cumulative cost of at most kT.

1.1.1 Aggregate Method

The aggregate method follows the most intuitively from the description given above. By employing the aggregate method, we essentially have to argue through the sequence of preceding operations required to perform a certain operation, add up the time taken to execute all of them, and then divide by the number of operations in the sequnce.

1.1.2 Potential Method

The potential is a much more powerful (but more unintuitive) method of amortized analysis. We define a potential function Φ that maps the state of our data structure to $\mathbb{R}$.

Suppose our data structure moves through states $D_0,\ D_1, \cdots, D_k$ over k operations. The potential energy of the data structure after the ith data structure is computed by simply applying Φ, such that $\Phi_i = \Phi(D_i)$. Additionally, we require that $\Phi_i \geq \Phi_0$ for all i — that is, we cannot "use" more energy than we started with.

Suppose the true cost of the ith operation was some quantity c_i. We define the *amortized* cost of the ith operation to be

$$\hat{c}_i = c_i + \Delta\Phi_i = c_i + (\Phi_i - \Phi_{i-1})$$

If we sum over the amortized costs,

$$\sum_{i=1}^{k} \hat{c}_i = \sum_{i=1}^{k} c_i + \Phi_k - \Phi_0$$

Recalling that $\Phi_k \geq \Phi_0$, we can conclude that

$$\sum_{i=1}^{k} \hat{c}_i \geq \sum_{i=1}^{k} c_i$$

so we can argue that the sum over all of the costs is upper-bounded by the sum over all of the amortized costs. Furthermore, the amortized cost will be upper bounded by

$$\frac{1}{k} \sum_{i=1}^{k} \hat{c}_i$$

so, if we are able to upper-bound $\hat{c}_i$, we can bound the overall amortized cost.

The challenge in using the potential method for analysis is coming up with Φ itself. This requires a lot of trial and error, but intuitively, we want $\Delta\Phi_i$ to be more negative for more expensive c_i's in order to offset their cost, balancing them with cheaper c_i's.

1.2 Competitive Analysis

Competitive analysis aims to quantify exactly how sub-optimally a deterministic algorithm will perform in an online setting, compared to the outputs of the *most optimal* algorithm of the same problem in an offline setting; we are, in some sense, comparing our results to an omniscient and powerful black box.

We say that a deterministic algorithm $\mathcal{A}$ is α-competitive for $\alpha \geq 1$ iff, for any sequence of requests R,

$$C_{\mathcal{A}}(R) \leq \alpha C_{OPT}(R) + c$$

where α can be any function of the input and c is a constant. $C_{\mathcal{A}}$ and C_{OPT} refer to the costs of $\mathcal{A}$ and the optimal algorithm OPT on input R, respectively.

We can extend this definition to randomized algorithms by discussing the *expected cost*:

$$\mathbb{E}[C_{\mathcal{A}}(R)] \leq \alpha C_{OPT}(R) + c$$

The potential method for amortized analysis provides us with a nice framework for proving a competitive bound. In particular, if we are able to express a valid potential function Φ that somehow encodes the differences between the operations of $\mathcal{A}$ and OPT at every request r_i, then we can bound the cost of $\mathcal{A}$ by the cost of OPT. In particular, if, while processing r_i, $\mathcal{A}$ costs c_i and OPT costs $c_i^\star$, we can claim:

$$\begin{aligned} C_{\mathcal{A}}(R) &= \sum_{i=1}^{k} c_i \\ C_{OPT}(R) &= \sum_{i=1}^{k} c_i^\star \\ \hat{c}_i &= c_i + \Delta\Phi_i \\ C_{\mathcal{A}}(R) &\leq \sum_{i=1}^{k} \hat{c}_i \end{aligned}$$

So if we can find a way to bound $\hat{c}_i$ by some $\alpha c_i^\star$, then we can make a claim about α-competitiveness.

1.3 Probabilistic Analysis

For the most part, we care about bounding the probability that an event will occur, or estimating the average value of some output of an algorithm. We can make use of the following tools:

1. **Union Bound:** For events $A_1, \cdots, A_n$,

$$\Pr\left[\bigcup_{i=1}^{n} A_i\right] \leq \sum_{i=1}^{n} \Pr[A_i]$$

2. **Linearity of Expectation:** For random variables $X_1, \cdots, X_n$,

$$\mathbb{E}\left[\sum_{i=1}^{n} X_i\right] = \sum_{i=1}^{n} \mathbb{E}[X_i]$$

3. **Markov's Inequality:** For a non-negative random variable X with $\mu = \mathbb{E}[X]$,
$$\Pr[X \geq a] \leq \frac{\mu}{a}$$

4. **Chebyshev's Inequality:** For a random variable X with $\mu = \mathbb{E}[X]$ and $\sigma^2 = \text{Var}[X]$,
$$\Pr[|X - \mu| \geq k] \leq \frac{\sigma^2}{k^2}$$

5. **Chernoff Bounds:** For $X = X_1 + \cdots + X_n$ with each X_i being a random variable taking values in $\{0, 1\}$ and $\mu = \mathbb{E}[X]$,
$$\Pr[X \geq (1+\beta)\mu] \leq e^{-\beta^2\mu/3} \text{ for } 0 < \beta \leq 1$$
$$\Pr[X \geq (1+\beta)\mu] \leq e^{-\beta\mu/3} \text{ for } 1 \leq \beta$$
$$\Pr[X \leq (1-\beta)\mu] \leq e^{-\beta^2\mu/2} \text{ for } 0 < \beta < 1$$

2 Data Structures

2.1 Union-Find

The Union-Find data structure keeps track of a set of *disjoint sets* of elements $\mathcal{S} = \{S_1, \cdots, S_r\}$. Each set has a representative element $\text{Rep}[S_i]$, which is an element $x \in S_i$ that "labels" S_i and its elements in operations that request specifying set membership. It supports the operations:

- Make-Set(x): Add the singleton set $\{x\}$ to the data structure, with $\text{Rep}[\{x\}] = x$.
- Find-Set(x): Return the representative element of the set x belongs to (i.e. find $\text{Rep}[S(x)]$).
- Union(x, y): If x and y are in different sets $S(x)$ and $S(y)$, merge them into one set $S(x) \cup S(y)$ and choose a representative element for that set.

Union-Find can be implemented efficiently with a forest-of-trees approach: each set is represented by a separate tree, with the root node being the representative element of the set of all elements in the tree. Furthermore, utilizing

a union-by-rank optimization (always merge the shorter tree into the taller tree) and a path-compression optimization (on each call to FIND-SET, update x such that its parent is the root node itself) leads to an implementation with an amortized cost of $O(\alpha(n))$, where α is the inverse Ackermann function.

2.2 Hash Tables

A DICTIONARY is a data structure that stores key-value pairs (x.KEY, x.VALUE). It supports the operations:

- INSERT(k, v): Add the pair (k, v) to the dictionary.
- DELETE(k): Remove the item x from the dictionary with x.KEY $= k$, if it exists.
- SEARCH(k): If the dictionary stores an x such that x.KEY $= k$, return x.VALUE, and return NULL otherwise.

A direct-accessing approach by storing items in memory addresses directly corresponding to their keys seems appealing here, as it allows for fast operations. However, depending on the range of the keys, such an approach may be extremely wasteful in terms of space. If we say that all of the keys are integers in the set $\{0, \cdots, |\mathcal{U}|-1\} = [|\mathcal{U}|]$, then the amount of space we need to allocate is $\Theta(|\mathcal{U}|)$; if $|\mathcal{U}| >> n$, this is suboptimal.

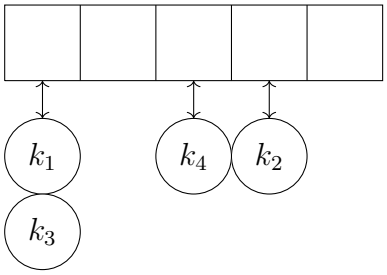
Instead, we'd like to shorten the stretch of $\mathcal{U}$ to an array of size m. We do so using a *hash function* $h : [|\mathcal{U}|] \to [m]$. Then, an item with key k is stored at the memory address corresponding to $h(k)$.

However, by the pigeonhole principle, there will necessarily be two distinct keys that map to the same hash value $h(k)$. We call these occurrences *collisions*. We'd like to have a scheme for dealing with collisions, if they occur, and we'd like to minimize the likelihood that they happen in the first place.

We note that if we define a *deterministic hash function* h, we always leave ourselves vulnerable to an adversary who can strategically choose keys such that many collisions will occur. Instead, we will talk about a hash family $\mathcal{H} = \{h : \mathcal{U} \to [m]\}$, from which we can randomly choose an h.

2.2.1 Chaining

The first approach to handling collisions is to simply store all of the keys with the same hash value in a linked list stored at location $h(k)$.



- INSERT(k, v): Add the pair (k, v) to the linked list at the address $h(k)$.
- DELETE(k): Remove the item x from the linked list at $h(k)$, if it exists.
- SEARCH(k): Return the value of x within the linked list at $h(k)$, if an x such that x.KEY $= k$ exists.

h is essentially partitions values in $[|\mathcal{U}|]$ into m different buckets. Any operation on an object with key k will first locate the linked list at address $h(k)$, and then traverse the list to find the correct node corresponding to the key. Ideally, we want the expected chain length to be small, so we do not have to traverse long chains in expectation.

We define the *load factor*

$$\alpha = \frac{n}{m}$$

such that the expected bucket size/chain length is α. In addition, we want to impose a universality assumption on $\mathcal{H}$: a hash family $\mathcal{H}$ is universal if, for all k_1 and $k_2 \in \mathcal{U}$ where $k_1 \neq k_2$,

$$\Pr_{h \in \mathcal{H}}[h(k_1) = h(k_2)] \leq \frac{1}{m}$$

The key assertion here is that the likelihood of a collision is low, and, in expectation, each key collides with α others.

If we ensure that h can be computed in constant time, all operations on the dictionary will require an $O(1)$ time calculation of $h(k)$, and then an expected $O(\alpha)$ time search through the chain linked to $h(k)$, for total runtime of $O(1 + \alpha)$ in expectation.

2.2.2 Open Addressing

Open addressing, by contrast, assigns a *probing sequence* $\sigma(k)$ to each element that stores addresses according to *priority*. Every sequence is some permutation of the addresses in the hash table. If an element encounters a collision at the first address in its probing sequence, it can move on to the next address in its sequence until it encounters a free spot.

Formally, for each $k \in \mathcal{U}$,

$$\sigma(k) = i_0^{(k)}, \cdots, i_{m-1}^{(k)}$$

where $i_j^{(k)}$ is k's jth most preferred placement in the hash table. $h(k,p) = i_p^{(k)}$, where p is the *probe number* (i.e. the index of the currently examined address in $\sigma(k)$).

- INSERT(k,v): Find the minimum p such that the value at $h(k,p)$ is NULL or DELETED, and insert (k,v) at $h(k,p)$.
- DELETE(k): Find the minimum p such that the key at $h(k,p)$ is k (if it exists). Replace $h(k,p)$ with DELETED.
- SEARCH(k): Find the minimum p such that the key at $h(k,p)$ is k (if it exists). Return the value at $h(k,p)$.

With chaining, we'd ideally want to keep the size of our buckets small. With open addressing, we instead want to minimize the number of probes we have to do in order to insert or find some element (i.e. minimize the p such that $h(k,p).\text{KEY} = k$ for all (k,v)). We enforce the *uniform permutation hashing assumption*, which is that $\sigma(k)$ is an independent and uniformly random permutation of the set $[m]$.

We can analyze the expected number of *unsuccessful* probes as a random variable X:

$$\begin{aligned}
\mathbb{E}[X] &= \sum_{p=1}^{m-1} p \cdot \Pr[X = p] \\
&= \sum_{p=1}^{m-1} \sum_{j=p}^{m-1} \Pr[X = j] \\
&= \sum_{p=1}^{m-1} \Pr[X \geq p]
\end{aligned}$$

$\Pr[X \geq p]$ can be estimated to be less than α^p, so that

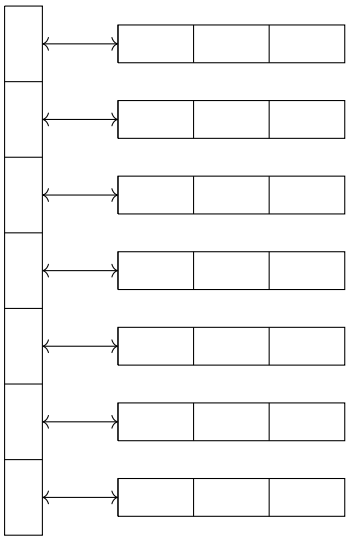
$$\mathbb{E}[X] \leq \sum_{p=1}^{m-1} \alpha^p < \frac{1}{1-\alpha}$$

Therefore, all operations with open addressing have an expected runtime of $O\left(\frac{1}{1-\alpha}\right)$.

2.2.3 Perfect Hashing

If we keep the size of the dictionary constant (i.e. maintain it as a static data structure) and focus only on SEARCH operations, it is possible to achieve a dictionary that can store every item without collisions in linear space; such a notion is known as a "perfect hash."

We note that, if we use $m \in O(n^2)$ space, we expect to be able to sample a hash function with no collisions with probability $\geq 1/2$ (and, on average, we expect to have to sample the function twice to obtain one without collisions). This doesn't solve the original problem, but such a a guarantee provides us with a path towards a solution. Instead of having a single hash table, we initialize two layers of hashing.



The large hash table samples an $h_1 \sim \mathcal{H}$, placing items into their respective buckets. Then, each sub-level hash table i randomly samples an $h_2 \sim \mathcal{H}$ that hashes all *collisions at its index* (rather than simply storing them in a linked list, like we see in chaining). The scheme is as follows:

1. Sample $h_1 \sim \mathcal{H}_n$, mapping $\mathcal{U} \to [n]$. Hash all keys to the first-layer hash table using h_1. Let n_i be the number of keys mapped to position i using h_1. If $\sum_{i \in [n]} > 4n$, resample h_1.

2. For each $i \in m$, pick a hash function $h_2 \sim \mathcal{H}_{m_i}$, mapping $\mathcal{U} \to [m_i]$ where $m_i \in \Theta(n_i^2)$. Hash each key in the hash table at index i using h_2. Resample h_2 if any collisions are detected.

As written, this scheme still uses $O\left(n + \sum_{i=1}^{n} n_i^2\right)$ space. However,

$$\mathbb{E}\left[\sum_{i=1}^{n} n_i^2\right] = n + \frac{2}{m}\binom{n}{2} < 2n$$

so the probability that the number of ordered pairs of collisions exceeds $4n$ is less than $\frac{1}{2}$, by Markov's inequality. Therefore, if we resample h_1 such that

$$\sum_{i=1}^{n} n_i^2 \leq 4n \in O(n)$$

then we can still guarantee that we use linear space.

3 Deterministic Polynomial-Time Algorithms

3.1 Greedy

A greedy approach to an optimization problem aims to *carve out* an optimal solution in a large search space by making *locally optimal* choices. Often, this requires fleshing out the notion of a "best choice" at any given stage, and a proof that making a series of local choices yields a globally optimal solution (i.e. we can ignore large parts of the search space without sacrificing correctness).

More formally, the correctness of a greedy algorithm rests on two key properties:

1. **Greedy Choice Property:** There exists an optimal solution that does contain the greedy choice at some stage.

2. **Problem Monotonicity:** If we split a problem into smaller subproblems, and then replace the solution to one of the subproblems with something at least as optimal, then the value of the overall solution is also at least as optimal.

- Applying this to the greedy choice: given any optimal solution that doesn't use the greedy choice, swapping in the greedy choice doesn't make the resulting solution *any worse*.
- This is closely related to the optimal substructure property (see 3.2).

The properties above can be arranged into an *exchange argument*, which follows the form of an inductive proof. Roughly, using the properties above, we can demonstrate a series of swaps between any optimal solution and the solution that applies the greedy choice that doesn't make the optimal solution any worse at any stage. Inducting on the number of choices made (which corresponds to the size of the problem), we can demonstrate that repeatedly applying the greedy choice will guarantee an optimal solution.

3.1.1 Interval Scheduling

Interval scheduling is a classic instance of a problem that can be solved with a greedy algorithm.

> We are given a series of n resource requests $R = \{r_i : i = 1, \cdots, n\}$ with each r_i specifying a time interval (a_i, b_i) for which they want to use a single resource. Two requests r_i, r_j are *mutually compatible* iff $r_i \cap r_j = \emptyset$. A subset of requests S is mutually compatible iff every pair in S is mutually compatible. We are asked to find the largest mutually compatible set.

At a high level, using a greedy algorithm for this problem means that it will follow the structure in *Interval Scheduling Sketch*.

Algorithm 1: Interval Scheduling Sketch

```
1 Set S ← ∅
2 while R is not empty do
3 |   Pick the greedy choice r ∈ R
4 |   If r is compatible with S, S ← S ∪ {r}
5 |   R ← R − {r}
6 end
7 return S
```

Once a ruleset for making a greedy choice is outlined, it must be shown to demonstrate the necessary properties for correctness. In particular, it turns out that choosing the remaining compatible request in R with the *earliest end time* is the best option. The exchange argument is as follows.

Let $S_G = \{s_j : 1 \leq j \leq m\}$ be the solution produced by repeatedly applying the greedy choice, and let $S_{OPT} = \{s'_j : 1 \leq j \leq m'$ be any optimal solution, placed in order of increasing end time.

Since each of these sets must be compatible, for any (a_j, b_j) in either set, $b_j \leq a_{j+1}$ for any index j.

Because S_{OPT} is an optimal solution, it follows that $m \leq m'$ — any solution differing from S_{OPT} must have the same size or be smaller, but it cannot be any larger (because this is a maximization problem).

Now, let $j^\star$ be the *earliest index* at which S_G disagrees with S_{OPT}, such that

$$S = \{s_1, \cdots, s_{j^\star-1}, s_{j^\star}, s'_{j^\star+1}, \cdots, s'_{m'}\}$$
$$S = \{s_1, \cdots, s_{j^\star-1}, s'_{j^\star}, s'_{j^\star+1}, \cdots, s'_{m'}\}$$

We claim the following:

1. If $j^\star > 1$, then $b_{j^\star-1} \leq a_{j^\star}$. This is because, by definition, $b_{j^\star-1} = b'_{j^\star-1}$, and $b_{j^\star-1} \leq a_{j^\star}$.

2. $b_{j^\star} \leq b'_{j^\star}$. We can assert this because we claimed that S_G and S_{OPT} *differed starting from* $j^\star$; if $b_{j^\star} > b'_{j^\star}$, then that means that the greedy algorithm would not have followed its own greedy choice, which is a contradiction.

3. In S_{OPT}, we can replace $s'_{j^\star}$ with $s_{j^\star}$ and maintain both optimality and compatibility. That is, $S = \{s_1, \cdots, s_{j^\star}, s'_{j^\star+1}, \cdots, s'_{m'}\}$ is an optimal solution.

   (a) $|S| = |S_{OPT}|$, as we have removed a single element from S_{OPT} and added back another element.

   (b) S is still compatible because, as stated in claim 2, $b_{j^\star-1} \leq a_{j^\star}$ and $b_{j^\star} \leq b'_{j^\star} \leq a'_{j_\star+1}$.

We can repeatedly apply the above swap until we obtain a solution S that is equally as optimal as S_{OPT}, with its prefix equal to the sequence provided by S_G.

$$S = \{s_1, \cdots, s_m, s'_{m+1}, \cdots, s_{m'}\}$$

The existence of $s'_{m+1}, \cdots, s'_m$ implies that there were compatible requests that started and ended beyond the last ending time in S_G, which means that the greedy algorithm would have had to terminate early and not have chosen any intervals past m. However, this is a contradiction: the greedy algorithm always chooses remaining compatible requests until none remain. Therefore, requests $s'_{m+1}, \cdots, s'_m$ cannot exist, and $m = m'$. Therefore, utilizing the greedy strategy produces a solution that is just as optimal as any other S_{OPT}.

Algorithm 2: Greedy Interval Scheduling

```
Sort R in order of increasing b_i
Set S ← ∅
while R is not empty do
    Pick the first r ∈ R
    If r is compatible with S, S ← S ∪ {r}
    R ← R − {r}
end
return S
```

3.1.2 Minimum Spanning Trees

A *tree* is a graph $T = (V, F)$ that is connected and has no cycles. Given a graph $G = (V, E)$, a *spanning tree* of G is a tree $T = (V, F)$ that contains all of the vertices of G, with $F \subseteq E$.

Given a graph $G = (V, E, w)$, with an edge weight mapping $w : E \to \mathbb{R}$, find a spanning tree $(T = V, F \subseteq E)$ that minimizes

$$w(T) = \sum_{e \in F} w(e)$$

We can approach this problem using a greedy strategy. In particular, we will greedily build up a minimum spanning tree T by repeatedly incorporating the next "safest edge" e, which is the edge with the minimum weight at any stage in our process such that introducing it will not cause a cycle in T.

> In particular, let $F \subseteq E$ be a strict subset of an MST. Let S be a cut such that there are no edges in F that cross the cut; i.e.,
>
> $$F(S, V\ S) = \{(u, v) \in F : u \in S, v \in V - S\} = \emptyset$$
>
> and let $e \in E(S, V - S)$ be of minimum weight. Then, the choice of edge e is safe to add to the MST, and $F \cup \{e\}$ is also a subset of an MST.
>
> We can make this claim because of the *exchange property of spanning trees*: if $G = (V, E)$ is a connected graph, $T = (V, F)$ is a spanning tree of G, and $e = (u, v) \in E - F$ is an edge not contained in T, then there exists a path P_e in T that connects the endpoints of e, u and v. Furthermore, $P_e \cup \{e\}$ forms a cycle, and the modified graph $T' = (V, F - \{e'\} \cup \{e\})$ is once again a tree $\forall e' \in F$.

Kruskal's algorithm iteratively grows a *forest* of several small trees, merging them as edges are introduced. It can be optimized with a Union-Find data structure. Prim's algorithm grows a *single tree*, adding edges in order of connectivity to the growing tree. It can be optimized using an efficient priority queue implementation, like a Fibonacci heap.

Algorithm 3: Kruskal's Algorithm

```
1 Initialize F = ∅
2 Initialize a Union-Find data structure U
3 E ← Sort(E, key=w)
4 for 1 ≤ i ≤ m do
5 |   if U.FindSet(u_i) ≠ U.FindSet(v_i) then
6 |   |   F ← F ∪ {e_i}
7 |   |   U.[Union](u_i, v_i)
8 |   end
9 end
```

Algorithm 4: Prim's Algorithm

```
1  Initialize F = ∅, P[u] = NULL for each u ∈ V, and an priority queue
     Q.
2  Pick an arbitrary s ∈ V. Q.INSERT(s, d(s) = 0).
3  For u ∈ V, u ≠ s, Q.INSERT(u, d(u) = ∞).
4  while Q ≠ ∅ do
5  |   u ← Q.DELETEMIN()
6  |   F ← F ∪ {(u, P[u])}
7  |   for v ∈ Q, (u, v) ∈ E do
8  |   |   If w(u, v) < d(v), Q.DECREASEKEY(v, d(v) ← w(u, v)), and
   |   |     update P[v] ← u.
9  |   end
10 end
```

3.2 Dynamic Programming

Dynamic programming is another algorithmic paradigm for optimization problems. It relies on the idea of *memoization*, or recording down solutions to all possible subproblems of a given start space and using this memo to compute solutions to larger problems.

Crucially, DP approaches differ from greedy algorithms in that *all* possible paths to an optimal solution are considered at once in DP, whereas a greedy algorithm ensures that it never has to backtrack on the choices it makes. The legwork of proving optimality in greedy algorithms is done up front with problem properties and restrictions (GCP, monotonicity), while DP is more akin to a (slightly optimized) exhaustive search of the solution space.

Solving a problem with dynamic programming requires that the problem exhibits the *optimal substructure property*, meaning that the optimal solution to a given problem can be constructed from optimal solutions to its smaller subproblems.

3.2.1 Weighted Interval Scheduling

The weighted interval scheduling problem is a variant of basic interval scheduling problem, but we now assign weights to each interval.

We are given a series of n resource requests $R = \{r_i : i = 1, \cdots, n\}$ with each r_i specifying a time interval (a_i, b_i) for which they want to use a single resource. In addition, each r_i is assigned a weight w_i. We are asked to find the maximum total weight that can be obtained by a mutually compatible set of chosen requests.

In some sense, the basic interval scheduling problem is a special case of the weighted interval scheduling problem: an instance of the original problem can be formulated as an instance to the weighted problem with all $w_i = 1$. In particular, any algorithm that solves weighted interval scheduling must also be able to produce a correct answer for the original problem. Taking this as inspiration, suppose we started with the structure imposed by the greedy choice for the original problem: namely, sorting R in order of increasing b_i.

Let $W(T)$ be the maximum total weight that can be achieved by taking requests of $T \subseteq R$, so that the answer to our original problem is just $W(R)$. Now, for each request r_i, we have a choice of either incorporating or not incorporating it into our final solution. We note that

$$W(R) = \max\{W(R - \{r_i\}), w_i + W(R - inc(r_i))\}$$

where $inc(r_i)$ denotes the set of requests in R that are incompatible with r_i, including r_i itself. The first term refers to not taking r_i, and the second term corresponds to taking r_i.

Now, consider r_n (which we've redefined to be the element in R with the latest end time): in particular, $inc(r_n)$ must be a *suffix* of the original array R; if t_n is the first index in the sorted R such that r_{t_n} is incompatible with r_n, then every element in the subarray $\{r_{t_n}, \cdots, r_n\}$ must also be incompatible with r_n. This allows us to prune large areas of our search space, and we can find t_n at any stage in $O(\log n)$ time due to binary search.

We can therefore "grow" a prefix of the total solution, iterating until we have considered all elements. In particular, for each $i = 1, \cdots, n$, we can compute

$$W(\{r_1, \cdots, r_i\}) = \max\{W(\{r_1, \cdots, r_{i-1}), w_i + W(\{r_j : j < t_i\})\}$$

There are n stages in this algorithm, each corresponding to computing

$$W(\{r_1, \cdots, r_i\})$$

If we use memoization, each max computation should only take constant time. Finding t_i for each i takse $O(\log n)$ time, and the initial processing of

sorting R takes $O(n \log n)$ time with an optimal comparison sorting algorithm (like merge sort). Therefore, this algorithm takes $O(n \log n)$ time in total.

3.3 Divide and Conquer

Divide-and-conquer algorithms follow three primary ideas:

1. **Divide:** Given a large problem, divide it a subproblems of size $n_1, \cdots, n_a$.
2. **Conquer:** Solve each subproblem recursively.
3. **Combine:** Combine the solutions to each subproblem into a solution to the original problem.

The runtime of a divide-and-conquer algorithm can be expressed as a recurrence relation, which can be solved with Master's theorem or the tree method.

$$T(n) = T(n_1) + \cdots T(n_a) + T_{divide} + T_{combine}$$

Often, each of the a subproblems will have the same size n/b, such that the recurrence becomes

$$T(n) = aT\left(\frac{n}{b}\right) + T_{divide} + T_{combine}$$

3.3.1 Rank-Finding

> Given an unsorted array A of n distinct numbers and a rank r, find the rth smallest element of A.

The observation underlying the divide-and-conquer approach to this problem is that we can perform a kind of "binary search" for the element at rank r. If we choose a pivot element $x \in A$ to split the array in half and compute

$$L(x) = \{y \big| y \in A, y < x\}$$
$$G(x) = \{y \big| y \in A, y > x\}$$

the sizes of the sets $L(x)$ and $G(x)$ tell us something about where we need to look for our answer. In particular, if $r = |L(x) + 1|$, then x is the rth smallest element, so we may just return x. If $r \leq |L(x)|$, our answer is in the set $L(x)$. Otherwise, our answer must be the element of rank $r - |L(x)| - 1$ in $G(x)$.

Now, we face the problem of choosing x in the first place. We would like to guarantee that x will split A into roughly balanced halves.

An element $x \in A$ is said to be c-balanced for $\frac{1}{2} \leq c < 1$ if $|L(x)|, |G(x)| \leq cn$. For the purposes of keeping the result of the runtime analysis linear, we would like our pivot to be $\frac{3}{4}$-balanced. We use a median-of-medians strategy:

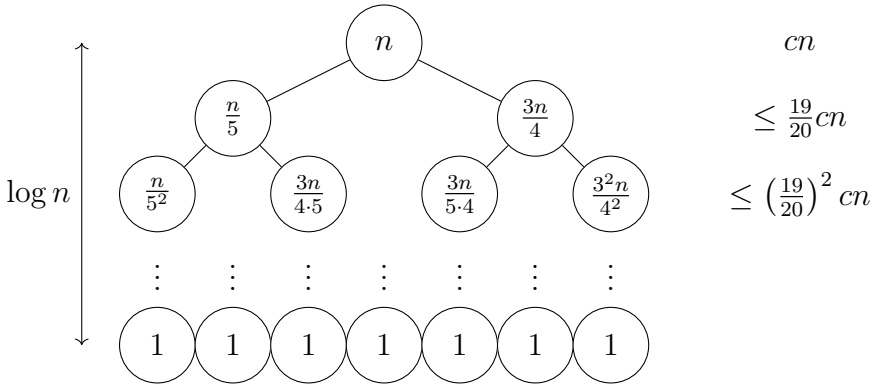
1. Divide A into $\lfloor n/5 \rfloor$ buckets of size 5, and find each of the bucket medians.
   - This takes linear time (as there are $O(n)$ buckets, and it takes $O(1)$ time to find the median of a constant-size bucket).
2. Given an array M of bucket medians (of size $\lfloor n/5 \rfloor$), call the overall rank-finding algorithm to find the median of M.
   - If the overall rank-finding algorithm takes $T(n)$ time, this step takes $T(n/5)$ time.

Algorithm 5: RankFind

```
1  If |A| = 1, return A[1].
2  Divide A into n/5 buckets, each of size 5. Let A_i denote the ith
     bucket.
3  M ← [Median(A_1), ··· , Median(A_{n/5})]
4  x ← RankFind(M, n/10)
5  Compute L(x), G(x).
6  if |L(x)| + 1 = r then
7  |  return x
8  end
9  if |L(x)| ≥ r then
10 |  return RankFind(L(x), r)
11 else
12 |  return RankFind(G(x), r − |L(x)| − 1)
13 end
```

The runtime of this algorithm is described by the recurrence

$$T(n) = T\left(\frac{n}{5}\right) + T\left(\frac{3n}{4}\right) + O(n)$$

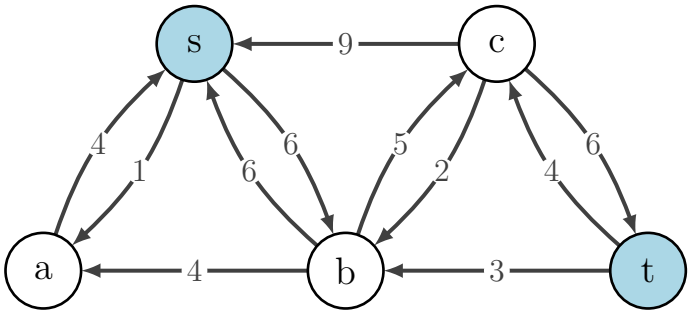


Each level contributes approximately $(19/20)^i$ work to the entire tree, which is a diminishing geometric series. We can therefore conclude that the sum of work across the tree is some constant times n, meaning that the algorithm takes $O(n)$ work. We can further verify this runtime using induction.

3.3.2 Fast Fourier Transform

3.4 Max-Flow and Min-Cut

A *flow network* is a directed graph G with a capacity function $c : E \to \mathbb{R}^+$ defined on its edges, a specified source s, and a specified sink t. We may "push" as much flow from s to t along the edges of G while staying within the capacity constraints for each edge.



Formally, on a network $G = \langle V, E, s, t, c \rangle$, a *net flow* is a function $f(u, v) \in \mathbb{R}$ that obeys the following:

1. **Capacity:** For all (u, v), $f(u, v) \leq c(u, v)$. If (u, v) are not directly connected by an edge in the graph, $c(u, v) = 0$.

2. **Flow conservation:** For every vertex u that is not s or t, the flow into the vertex matches the flow out of the vertex.

$$\sum_{v:(u,v)\in E} f(u,v) = \sum_{v:(v,u)\in E} f(u,v)$$

3. **Skew-symmetry:** $f(u,v) = -f(v,u)$ for all (u,v) pairs.

A mapping f that satisfies the above for a given network is said to be *feasible*. In addition, we define the *value* of a flow to be the total flow exiting the source vertex.

$$|f| = \sum_{v\in V} f(s,v)$$

For a flow f, we define its support $\text{supp}_f(G)$ as the subgraph containing all edges with strictly positive flow. The **flow decomposition lemma** states that $\text{supp}_f(G)$ can be decomposed into a collection of $s-t$ flow paths and flow cycles. In addition, the **flow integrality theorem** guarantees that, for a network in which all capacities are integers, there exists an integral maximum flow. Furthermore, the flow pushed along any edge will also be an integer.

3.4.1 Duality

There is a *dual* relationship between flows in a network and cuts in the underlying graph. An $s-t$ cut in a graph is a partition of its vertices into non-empty subsets S and $T = V/S$ such that $s \in S$ and $t \in T$. Further, the net flow across S is the total flow contributed by edges that *cross the cut*:

$$f(S) = f(S,T) = \sum_{u\in S}\sum_{v\in T} f(u,v)$$

Similarly, the capacity of the cut S is the total capacity of the edges crossing the cut.

$$c(S) = c(S,T) = \sum_{u\in S}\sum_{v\in T} c(u,v)$$

Observe that $f(S) \leq c(S)$ for any cut S (or else, we would violate the capacity constraints). In addition, we note that $f(S)$ must be the same across *any* $s-t$ cut by flow decomposition. We can conclude that any flow $f(S)$ is thus

upper-bounded by the smallest $c(S)$. If $S^\star$ is the minimum cut of the graph, then, for any flow f,

$$|f| \leq c(S^\star)$$

This is the *weak duality* between max-flow and min-cut. We can state a stronger result, called the *strong duality*: if $f^\star$ is a maximum-valued flow,

$$|f^\star| = c(S^\star)$$

In sum, the **max-flow min-cut theorem** states that, for a flow f in G, the following are equivalent:

1. $|f| = c(S)$ for some $s - t$ cut $(S, V/S)$.
2. f is a maximum flow.
3. G_f contains no augmenting paths.

3.4.2 Max-Flow

The max-flow problem asks the following:

> Given a flow network $G = \langle V, E, s, t, c \rangle$, find a flow $f^\star$ of *maximum value*.

The basic approach towards solving this problem is to come up with *some flow* f, and revising it repeatedly by adding paths to account for any missing flow, if such paths exist.

Define $c_f(u, v)$ as the residual capacity of edge (u, v) with respect to f:

$$c_f(u, v) = c(u, v) - f(u, v)$$

Define G_f as the residual flow graph of G with respect to f with capacity function c_f. G_f will contain all of the vertices of G, and the edges (u, v) such that $c_f(u, v) > 0$. An *augmenting path* is an $s - t$ path in G_f with positive residual capacity; we can push more flow along the augmenting path to f to produce a new flow of greater value. Provably, if we can find no augmenting paths in G_f, then we cannot increase the total flow.

The Ford-Fulkerson algorithm follows exactly this basic approach. With integral capacities, Ford-Fulkerson takes worst case $O(|f^\star||E|)$ time, where $|f^\star|$ is the largest flow that can be attained in the network. We can estimate

Algorithm 6: Ford-Fulkerson

```
Initialize f ← 0. while there is an s − t path P in G_f do
    Augment f along P by adding flow up to the bottleneck capacity
      c_f(P) = min_{e∈P} c_f(e)
end
return f
```

this *very loosely* as $C|V|$ if we are given that edges have capacity of at most C, which gives us a runtime of $O(C|V||E|)$.

We can make a few optimizations on this procedure. Instead of finding *any augmenting path* in a given iteration, the Edmonds-Karp algorithm finds the path that uses the fewest edges by running a BFS per iteration. This optimization yields a runtime of $O(|E|^2|V|)$ time.

If we instead attempt to find the augmenting path with the largest residual bottleneck capacity at every iteration, we would need to conduct a binary search over the remaining residual capacities along any $s-t$ path. This yields a runtime of $O(|E|^2 \log |V| \log |f^\star|) = O(|E|^2 \log |V| \log C|V|)$.

3.4.3 Min-Cut

If we have a max flow $f^\star$, we may find a minimum cut fairly easily: we first produce $G_{f^\star}$, and then run a a reachability test from s. All vertices reachable from s in $G_{f^\star}$ will be part of a minimum cut. This cut cannot contain t, or $f^\star$ would not be a max-flow by the max-flow min-cut theorem.

4 Randomization

Randomized algorithms fall into one of two categories: Monte Carlo and Las Vegas. Monte Carlo algorithms are guaranteed to run in polynomial time, but may be incorrect (with ideally low probability), whereas Las Vegas algorithms are guaranteed to be correct, but run in only expected polynomial time (and may be extremely slow in unlucky instances).

With Monte Carlo algorithms, we only need to provide a guaranteed upper bound on the error probability of the algorithm ($\Pr[\text{incorrect result}] \leq p$). We can then perform a process of *amplification*, through which the algorithm is run multiple times and the results are aggregated through some voting

scheme — this will result in an error probability of some p^i, so the success rate can be stated as $1 - p^i$.

4.1 Selecting a pivot

Many algorithms depend on choosing a good *pivot element* that partitions the input space into roughly equally-sized subsets to run divide-and-conquer algorithms (for example, RankFind and QuickSort). While we showed a deterministic algorithm to find a $\frac{3}{4}$-balanced element for RankFind, we can also select this pivot *randomly* and check if this pivot is a $\frac{3}{4}$-balanced element, and proceed with the rest of the algorithm. Although this step still requires $\Theta(n)$ time, it is much simpler than the deterministic median-of-medians method.

4.2 Markov Chains and Random Walks

A Markov chain is a sequence of random variables X_t that is *memoryless*, meaning that each state in the sequence X_{t+1} depends only on its direct predecessor X_t. For now, we consider X_t as a discrete random variable that can take on n states. A Markov chain $\{X_t : t \in \mathbb{N}\}$ is *time-homogeneous* if there exists an $(n \times n)$ transition matrix W such that, $\forall t \in \mathbb{N}$ and all pairs of states (u, v),

$$\Pr[X_{t+1} = v | X_t = u] = W_{uv}$$

That is, W stores all probabilities associated with state transitions.

There is an equivalent graphical way of looking at things. From a time-homogeneous Markov chain, we may construct a graph $G_W = (V, E, w : E \to \mathbb{R})$ with vertices corresponding to the states of the Markov chain and edges corresponding to state transitions. In addition, for every transition (u, v) with probability W_{uv}, we may assign

$$w(u, v) = W_{uv}$$

A random walk on G_W is equivalent to the evolution of the corresponding time-homogeneous Markov chain X_t.

Given W, we can describe the probability distribution π_t of the t'th time

step X_t in terms of the initial distribution π_0. Observe that

$$\begin{aligned}\Pr[X_t = x_t] &= \sum_{x_{t-1}} \Pr[X_t = x_t, X_{t-1} = x_{t-1}] \\ &= \sum_{x_{t-1}} \Pr[X_t = x_t | X_{t-1} = x_{t-1}] \Pr[X_{t-1} = x_{t-1}] \\ &= \sum_{x_{t-1}} \pi_{t-1}(x_{t-1}) W_{x_{t-1}, x_t}\end{aligned}$$

Composing the recurrence relation, we find that

$$\pi_t = \pi_0 W^t$$

where π_t is a $(1 \times n)$ row vector containing the probability that X_t takes on a certain state at step t.

4.2.1 Convergence

A distribution $\pi_\star$ is *stationary* with respect to W if

$$\pi_\star = \pi_\star W$$

i.e., applying a transition does not change the probability distribution over states.

We may observe that taking progressively higher powers of W yields a matrix with rows identical to each other; if this happens, this row is equivalent to the stationary distribution. Such a case is termed *convergence*. If W converges, then for any two starting points π_0 and π_0',

$$\lim_{t \to \infty} \pi_0 W^t = \lim_{t \to \infty} \pi_0' W^t = \pi_\star$$

4.2.2 Connectivity

Two vertices in a graph u and v communicate with each other ($u \leftrightsquigarrow v$) if there is a directed path from u to v and vice versa. $\leftrightsquigarrow$ forms an equivalence relation over the vertices, with equivalence classes defined by the strongly connected components of the graph. The induced directed graph representing strongly connected components (the condensation graph) must be acyclic. A component is transient if it has an outgoing edge, and is recurrent otherwise.

The period of a vertex v is the greatest common divisor of the lengths of all directed cycles in its strongly connected component. v is periodic if its period is strictly > 1, and is aperiodic otherwise.

An undirected graph may only take periods of 1 or 2, as every vertex connected via an edge automatically participates in a length-2 cycle. An undirected connected graph has a periodic vertex iff it is bipartite.

A vertex is automatically aperiodic if it has a self loop. A Markov chain is *lazy* if all vertices have self loops.

4.2.3 Reversibility

A stochastic process is reversible if walking both forwards and backwards through the process looks the same from a probabilistic viewpoint — that is, a sequence of states undertaken in the forward and backward direction would carry the same statistics. A time-homogeneous Markov process defined by a matrix W is reversible iff there exists a stationary distribution $\vec{\pi}$ such that

$$\pi_x W_{xy} = \pi_y W_{yx}$$

otherwise known as the detailed balance conditions.

4.2.4 Fundamental Theorem of Markov Chains

Every time-homogeneous Markov chain W has a stationary distribution.

- If the underlying directed graph is *strongly connected*, then the stationary distribution is *unique*. (A weaker notion is that if the graph has exactly one recurrent component, then this stationary distribution must be unique).
- If all vertices in the recurrent components are aperiodic, then for every choice of initial distribution π_0, the distribution $\pi_t = \pi_0 W^t$ of X_t converges to some stationary distribution as $t \to \infty$.

For any transient component $\mathcal{C}$, $\Pr[X_t \in \mathcal{C}] \to 0$ as $t \to \infty$.

4.2.5 Metropolis-Hastings

Monte Carlo Markov Chain (MCMC) algorithms are designed for the task of efficiently sampling from a (possibly extremely complicated) probability

distribution by constructing a Markov Chain with a stationary distribution equivalent to the desired probability distribution.

In particular, the Metropolis-Hastings algorithm deals with sampling from distributions defined by some function $\bar{v} : \mathcal{D} \to \mathbb{R}$, where the probability of some event X is proportional to $\bar{v}(X)$, up to some scaling factor K:

$$K = \sum_{x \in \mathcal{D}} \bar{v}(x)$$

Thus, if v is the intended probability distribution, then $\bar{v} = Kv$. In practice, K may be very computationally difficult to compute directly, especially if $\mathcal{D}$ is large. For the purposes of M-H, we assume we have easy access to the function $\bar{v}$ itself.

Metropolis-Hastings takes a "proposed" distribution $g : \mathcal{D}^2 \to \mathbb{R}$ that is comparatively easier to sample from, and transforms it into the desired distribution by designing a reversible Markov Chain according to g and $\bar{v}$. When g is sampled to obtain a proposed next state according to the current state, we may either accept or reject this proposal based on the values of $\bar{v}$. In particular, for all $x \neq y \in \mathcal{D}$,

$$W_{xy} = g(y|x) \cdot p_{acc}(y, x)$$

such that

$$p_{acc}(y, x) = \min \left\{ 1, \frac{\bar{v}(x)g(x|y)}{\bar{v}(y)g(y|x)} \right\}$$

and

$$W_{xx} = 1 - \sum_{y \neq x} W_{xy}$$

5 Online Algorithms

An online algorithm deals with a fundamentally different problem setting. Instead of being given the entire input to a problem at the beginning, we are instead given the input in smaller pieces at a time: we must then decide what to do to produce some optimal solution on the fly, knowing that we cannot revisit and revise our decisions.

An online problem is inherently harder than its offline version, because we have access to less information at every stage (we cannot see the future).

In particular, for any deterministic algorithm that aims to solve an onlin problem, we can envision an adversarial input sequence (which is specifically designed around the deterministic algorithm's procedures to yield poor performance). For this reason, it isn't very interesting to reason about the *worst case* running time for any such algorithm; instead, we can analyze the running time *in expectation* (across randomized input sequences), or compare the performance of such an algorithm to some optimal offline algorithm.

5.1 Online Interval Scheduling

We modify the interval scheduling algorithm, such that instead of being given the entire set R at once, we are given one request r_i at a time. At each time step, we must choose whether to keep or discard r_i immediately, and we cannot backtrack on our decisions.

Consider the greedy procedure that just takes requests as they come if they are compatible with the existing solution set. We note that the best

Algorithm 7: Greedy Online Interval Scheduling

```
Set S ← ∅
while request r is received do
    if r compatible with S then
        S ← S ∪ {r}
    end
end
return S
```

possible performance of this algorithm is when the order of the requests aligns with the order of the processing in the offline greedy algorithm — that is, when r is sent in in order of increasing endpoints b. Otherwise, this algorithm will perform sub-optimally on the entire set R.

If we enforce that requests are sent in a uniformly random order from some fixed problem instance R, we can begin to analyze the expected performance of this algorithm.

Since we are interested size of our output S, we define $X = |S|$ be the random variable that corresponds to the size of S. We want to find $\mathbb{E}[X]$ to gauge our average performance.

Now, define indicator variables X_i corresponding to the ith request r_i:

$$X_i = \begin{cases} 1 & r_i \in S \\ 0 & r_i \notin S \end{cases}$$

such that

$$X = \sum_{i=1}^{n} X_i$$

By linearity of expectation,

$$\mathbb{E}[X] = \sum_{i=1}^{n} \mathbb{E}[X_i]$$

and because each X_i is an indicator random variable,

$$\mathbb{E}[X_i] = \Pr[X_i = 1]$$

Now, we can analyze the problem in terms of incorporating a single request.

$$\Pr[X_i = 1] \geq \Pr[r_i \text{ arrives first out of } inc(r_i)] = \frac{1}{|inc(r_i)|}$$

such that

$$\mathbb{E}[X] \geq \sum_{i=1}^{n} \frac{1}{|inc(r_i)|}$$

Now, the details of $|inc(r_i)|$ depend on the specific problem instance R. However, the above analysis provides us with an approximation of the optimality of our algorithm for a given set of requests.

5.2 Self Organizing Lists

A self-organizing list L holds n distinct elements that supports Access(x), which finds and returns the element x. The cost of accessing x is equal to how far x is stored from the first element of L. L may perform any transpositions (swapping of adjacent elements) after a call to Access, and each transposition costs 1.

A good algorithm will be able to perform transpositions at optimal times such that subsequent accesses will be cheaper. Generally, any deterministic

algorithm $\mathcal{A}$ will perform t_i transpositions after processing the ith access request, resulting in a state L_i. The cost of $\mathcal{A}$ will be

$$C_{\mathcal{A}}(R) = \sum_{i=1}^{k} [\textsc{Rank}_{L_{i-1}}(x_i) + t_i]$$

It turns out that the Move-to-Front (MTF) strategy, where items are transposed to the front of the list after they are accessed, is 4-competitive with any optimal strategy for this problem.

Let c_i be the cost of the MTF strategy on the ith access, and let $c_i^\star$ be the cost of the OPT strategy on the same access. Let r_i be the rank of the ith access on the previous list state (that is, $\textsc{Rank}_{L_{i-1}}(x_i)$). Let t_i be the number of transpositions performed by OPT.

$$c_i = 2 \cdot r_i - 1$$
$$c_i^\star = r_i^\star + t_i$$

Now, define $\textsc{Inv}(L_i, L_i^\star)$ as the number of pairs (x, y) such that x comes before y in L_i, but they are reversed in $L_i^\star$ — we call this an *inversion*. We define

$$\Phi = 2 \cdot |\textsc{Inv}(L_i, L_i^\star)|$$

For every new request, we can analyze the performance of this MTF strategy as compared to the (hypothetical) OPT strategy by bounding

$$\hat{c_i} = 2r_i - 1 + \Delta\Phi$$

By doing casework on the scenarios in which Φ would change (i.e. changing the order of elements), we find that

$$\Delta\phi \leq 2(|A_i| - |B_i| + t_i)$$

where A_i is the set of elements that are before the ith request x_i in L_{i-1} and $L_{i-1}^\star$, and B_i is the set of elements before x_i in L_{i-1} and after it in $L_{i-1}^\star$. It can be shown that

$$\hat{c}_i \leq 4(r_i^\star + t_i) = 4c_i$$

so this strategy is 4-competitive.

Note that we did not *specify* an optimal algorithm here, nor did we have to argue about the optimality of any particular algorithm for this process; we just abstracted the behavior of *some* hypothetical algorithm into its descriptive quantities, and showed 4-optimality by reasoning through each respective algorithm's decisions step-by-step.

5.3 Online Learning

Online learning is concerned with *supervised learning tasks*: given a stream of input data with corresponding labels, it is the goal of an online learning scheme to predict a value or action such that some (possibly adversarial) cost function is minimized. The scheme is permitted to revise its weights upon learning that it is right or wrong at any step.

Formally, we can segment our timeline into time steps indexed by t. Let A be the set of all actions that can be chosen, and let $a \in A$ be a given action. We define a cost function per-time-step, such that $c_t(a) \in [-1, 1]$ is the cost incurred by action a at time t. Let p_t be a probability distribution over A, which we use to sample a.

The expected loss incurred by our decision at a given time step is given by

$$\mathbb{E}[C_t] = \sum_{a \in A} p_t(a) c_t(a)$$

Over T time steps,

$$\mathbb{E}[C] = \sum_{t=0}^{T} \mathbb{E}[C_t] = \sum_{t=0}^{T} \sum_{a \in A} p_t(a) c_t(a)$$

For the sake of analysis, we would like to compare ourselves to some measure of performance. We call this a *benchmark*, given by the random variable B. Our *regret* after T time steps is

$$\frac{\mathbb{E}[C] - \mathbb{E}[B]}{T}$$

We have vanishing regret with respect to the benchmark B if

$$\lim_{T \to \infty} \frac{\mathbb{E}[C] - \mathbb{E}[B]}{T} = 0$$

Provably, no algorithm can have vanishing regret with respect to B such that $B = \sum_{t=0}^{T} \min_{a \in A} c_t(a)$ — that is, the benchmark that expects the most optimal choice made at every time step. However, it *is* possible to achieve vanishing regret with respect to the benchmark that makes the *best fixed choice*: $B = \min_{a \in A} \sum_{t=0}^{T} c_t(a)$. Regret with respect to this benchmark is called external regret. While it is not possible for any deterministic algorithm to achieve vanishing external regret, it is possible for a randomized algorithm to do so.

5.3.1 Weighted Majority

Imagine that, as a learner, we have n experts, each initially assigned a weight $w_i^0 = 1$. Each of these experts may output a certain prediction out of $\{0, 1\}$. We output our own prediction by taking the weighted majority of the experts' predictions.

The adversary subsequently gives us feedback about our prediction. We may then re-weight the experts accordingly. For each expert i:

$$w_i^{t+1} = \begin{cases} w_i^t & \text{if this expert predicted correctly} \\ (1-\varepsilon) w_i^t & \text{if this expert was wrong} \end{cases}$$

for some predefined parameter ε.

We can bound the number of mistakes made by this procedure by the number of mistakes made by the *best expert* (i.e., a fixed expert that made the least number of mistakes over all rounds). Let m be the number of mistakes made by the Weighted Majority procedure, and let $m^\star$ be the number of mistakes made by the best expert. Then,

$$m \leq 2(1+\varepsilon) m^\star + \frac{2 \ln n}{\varepsilon}$$

This analysis is achieved by considering the sum of all weights at a given time step. Let

$$W^t = \sum_{i=1}^{n} w_i^t$$

We observe that weights can only *decrease*, so $W^{t+1} \leq W^t$ for all t. In addition, every time our *learner* makes a mistake, this means that over half

of the weights must be decreased by a factor of $(1-\varepsilon)$. Thus, every time we make a mistake,

$$\begin{aligned} W^{t+1} &= \sum_{i\text{: correct}} w_i^t + (1-\varepsilon) \sum_{i\text{: incorrect}} w_i^t \\ &= \sum_{i=1}^{n} w_i^t - \varepsilon \underbrace{\sum_{i\text{: incorrect}} w_i^t}_{\geq W^t/2} \\ &\leq \left(1 - \frac{\varepsilon}{2}\right) W^t \end{aligned}$$

Thus, if m mistakes are made over T total time steps, then

$$W^T \leq \left(1 - \frac{\varepsilon}{2}\right)^m W^0 = \left(1 - \frac{\varepsilon}{2}\right)^m n$$

Consider that the weight of the best expert $i^\star$ must be

$$w_{i^\star}^t = (1-\varepsilon)^{m^\star}$$

and it is trivially less than W^T.

$$\begin{aligned} (1-\varepsilon)^{m^\star} \leq W^T &\leq \left(1 - \frac{\varepsilon}{2}\right)^m n \\ (1-\varepsilon)^{m^\star} &\leq \left(1 - \frac{\varepsilon}{2}\right)^m n \\ m^\star \ln(1-\varepsilon) &\leq m \ln\left(1 - \frac{\varepsilon}{2}\right) + \ln n \end{aligned}$$

Since $-x - x^2 \leq \ln(1-x) \leq -x$ for $x \in [0, 1/2]$,

$$\begin{aligned} -m^\star(\varepsilon + \varepsilon^2) &\leq -\frac{m\varepsilon}{2} + \ln n \\ \frac{m\varepsilon}{2} &\leq m^\star\varepsilon(1+\varepsilon) + \ln n \\ m &\leq 2m^\star(1+\varepsilon) + \frac{2\ln n}{\varepsilon} \end{aligned}$$

so the external regret is

$$m - m^\star \leq m^\star(1+2\varepsilon) + \frac{2\ln n}{\varepsilon}$$

As $m^\star$ may grow as $\Theta(T)$, this deterministic algorithm cannot achieve vanishing external regret.

5.3.2 Multiplicative Weights

The Multiplicative Weights scheme is a randomization of the Weighted Majority procedure — instead of making a decision deterministically based on the weighted majority of the experts, we instead sample an expert from a probability distribution defined by the weights assigned to them, and output their answer.

For a time step t, let M^t be the random variable corresponding to the number of mistakes made at that time step. Then,

$$\begin{aligned} W^{t+1} &= (1 - \varepsilon\mathbb{E}[M^t])W^t \\ &\leq W^t e^{-\varepsilon\mathbb{E}[M^t]} \end{aligned}$$

Let $M = \sum_t^T M_t$. Similar to the analysis for the Weighted Majority procedure,

$$\begin{aligned} (1-\varepsilon)^{m^\star} &\leq e^{-\varepsilon\mathbb{E}[M]}n \\ m^\star \ln(1-\varepsilon) &\leq -\varepsilon\mathbb{E}[M] + \ln n \\ \mathbb{E}[M] &\leq (1+\varepsilon)m^\star + \frac{\ln n}{\varepsilon} \end{aligned}$$

6 Linear Programming

A *linear programming problem* (LP) concerns the optimization of a linear objective function subject to linear constraints. For example,

$$\begin{aligned} \max \quad & \sum_{i=1}^{n} c_i x_i \\ \text{s.t.} A_{1,1}x_1 + \cdots A_{1,n}x_n &\geq b_1 \\ &\vdots \\ \text{s.t.} A_{i,1}x_1 + \cdots A_{i,n}x_n &= b_i \\ &\vdots \\ \text{s.t.} A_{m,1}x_1 + \cdots A_{m,n}x_n &\leq b_m \\ x_1, \cdots, x_n &\geq 0 \end{aligned}$$

is a *maximization* LP with variables $x_1, \cdots, x_n$. The *standard form* for a maximization LP contains only $\leq$ constraints and non-negativity constraints

for every variable. Further, note that we may use matrix-vector notation for conciseness:

$$\begin{aligned} \max\ & \vec{c}^T \vec{x} \\ A\vec{x} &\leq \vec{b} \\ \vec{x} &\geq \vec{0} \end{aligned}$$

A standard *minimization* LP looks like

$$\begin{aligned} \min\ & \vec{c}^T \vec{x} \\ A\vec{x} &\geq \vec{b} \\ \vec{x} &\geq \vec{0} \end{aligned}$$

A few notes on conversion to standard form:

1. Any $\leq$ constraint can be converted into a $\geq$ constraint by negating all coefficients involved, and vice versa.
2. An $=$ constraint is equivalent to upholding both a $\leq$ and $\geq$ constraint.
3. If a variable x is unconstrained, it can be written as a difference $x^+ - x^-$, where $x^+, x^- \geq 0$.
4. If $x \leq 0$, we can define a new variable $x' = -x$ and replace all instances of x with x', using appropriate negations of coefficients.

A solution $\vec{\tilde{x}}$ is *feasible* if it satisfies all of the constraints. The *feasible region* of the LP is the set of all its feasible solutions. We say that the feasible region is *bounded* if it fits into an n-dimensional ball.

An optimal solution to the LP is some $\vec{x^\star}$ such that

$$\vec{c}^T \vec{x^\star} \geq \vec{c}^T \vec{\tilde{x}}$$

for all feasible solutions $\vec{\tilde{x}}$. An LP itself is *bounded* if this optimal value is finite, and is unbounded otherwise.

6.1 Duality

We can provide guarantees about the existence and tightness of solutions to an LP by reasoning about another, related linear program, called its *dual*. In particular, for a *primal* LP $\mathcal{P}$, for example,

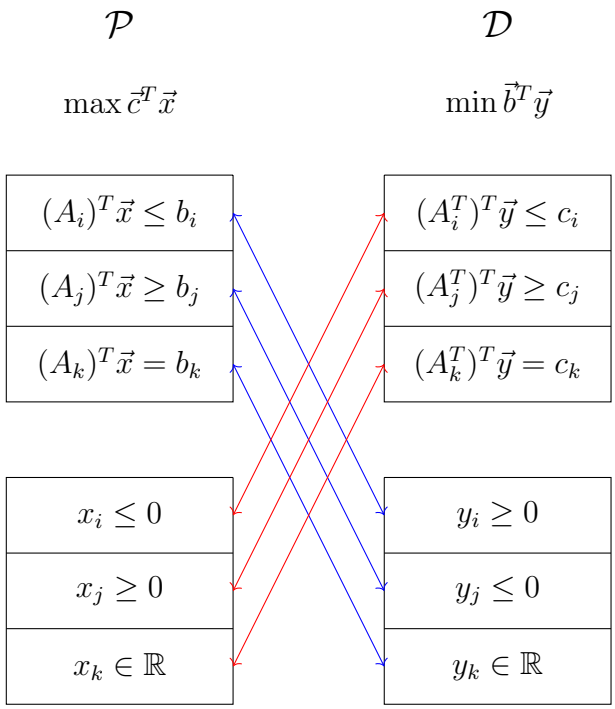
$$\begin{aligned} \mathcal{P} : \max \vec{c}^T \vec{x} \\ A\vec{x} \leq \vec{b} \\ \vec{x} \geq \vec{0} \end{aligned}$$

we may find its dual $\mathcal{D}$

$$\begin{aligned} \mathcal{D} : \min \vec{b}^T \vec{y} \\ A^T \vec{y} \geq \vec{c} \\ \vec{y} \geq \vec{0} \end{aligned}$$

The variables in $\mathcal{D}$ correspond to the constraints in $\mathcal{P}$, and the variables in $\mathcal{P}$ similarly correspond to the constraints in $\mathcal{D}$. These constraints and variable assignments are found, essentially, by taking the constraints from $\mathcal{P}$ and adding *multipliers* y_i to reason about the *largest possible value* $\mathcal{P}$ can take; we are interested in upper-bounding the maximization problem. If we can reason about such a value, provided it exists, then we may be convinced of the optimality of some candidate solution $\vec{x^\star}$.

The conversions we need to perform to take a dual are outlined like so:



6.2 Complementary Slackness

Complementary slackness provides a relationship between a variable in a primal LP and its corresponding dual constraint. Formally, for optimal primal solutions $\vec{x}^\star$ and dual solutions $\vec{y}^\star$,

$$\vec{x}^\star \cdot (A^T \vec{y}^\star - \vec{c}) = \vec{0}$$

where $\cdot$ refers to a component-wise product. Essentially, if a primal variable takes on a *nonzero value* (meaning that it contributes in some way to the optimal value), then its corresponding constraint doesn't necessarily have to be tight; conversely, if the corresponding dual constraint is *not tight*, then the variable must be 0, and thus cannot contribute to the optimal primal value.

7 Approximation

Some intractable problems, though difficult to solve exactly, may admit algorithms that get "close enough" to a correct solution (for some formal measure

of "close"). Such algorithms are called *approximation* algorithms. Formally, for some instance P of a problem $\mathcal{P}$ with a proposed solution x, we say that some a function $Q_P(x)$ determines the *quality* of the solution x on P (informally, how "good" of a solution x is). For optimization problems, Q_P can just be defined as the value of the objective function of the problem on x; how good of a solution x is will be determined by how far it is from achieving the optimum value.

Let $\mathrm{OPT}(P)$ denote the optimum value achievable on a problem instance P, and for any algorithm $\mathcal{A}$, let $\mathcal{A}(P)$ denote the value achieved by $\mathcal{A}$ on P.

If $\mathcal{P}$ is a *minimization* problem, $\mathcal{A}$ is an α-approximation on $\mathcal{P}$ for $\alpha \geq 1$ iff $\forall P \in \mathcal{P}$,

$$\mathcal{A}(P) \leq \alpha \cdot \mathrm{OPT}(P)$$

that is, $\mathcal{A}$ does better than *some multiple* of the optimum target. Symmetrically, if $\mathcal{P}$ is a *minimization problem*, then $\mathcal{A}$ is an α-approximation iff

$$\mathrm{OPT}(P) \leq \alpha \cdot \mathcal{A}(P)$$

α can be defined in a unified way across minimization and maximization problems as

$$\alpha = \max\left(\frac{\mathrm{OPT}(P)}{\mathcal{A}(P)}, \frac{\mathcal{A}(P)}{\mathrm{OPT}(P)}\right)$$

Similar to competitive analysis, determining the approximation ratio α does not depend on *knowing* the optimum solution (if we knew what it was, we would not need an approximation in the first place). Rather, we can perform a counting argument.

7.1 Greedy Approximation

Often, the most intuitive greedy algorithms will provide surprisingly favorable approximation ratios; it is just a matter of analyzing their performance. Generally, an analysis of optimality will require the following:

1. Define a correspondence between elements of a hypothetical optimal solution and the greedy solution.

2. Relate the size of the optimal solution and the size of the greedy solution by reasoning about the factors involved in the many-to-one correspondence.

3. Even though we do not have access to the optimal procedure, we do know that any greedy algorithm will not have any "slack" remaining (that is, it will never terminate without having chosen everything that it could have).

7.2 LP Rounding

Some combinatorial optimization problems may be represented as Integer Linear Programs (ILPs). By itself, this technique does not yield anything more than a certain kind of formalization, as arbitrary ILPs are NP-hard to solve. However, we may relax them into regular linear programs by removing integrality constraints, and then decide on some rounding scheme to produce a final solution. While the solution may not be exact, it may achieve some level of proximity to the optimum value. We may use the following properties to determine approximation bounds:

1. If P is the original optimization problem formulated as an ILP, and P_{LP} is the relaxation of the LP, then we note that any feasible solution for P is also a feasible solution for the ILP.
2. Thus, the optimal solution to P must be "smaller" in value compared to the optimal solution to P_{LP}.
3. If we can find a relationship between the value of the *rounded* solution to P_{LP} and $\mathrm{OPT}(P_{LP})$ by some factor α, then we may extend this to a relation with $\mathrm{OPT}(P)$.

7.3 Randomized Approximations

If $\mathcal{A}$ is a randomized algorithm, we say that it achieves an approximation ratio $\alpha \geq 1$ if it compares to an optimal solution *in expectation*. That is,

$$\alpha = \max\left(\frac{\mathrm{OPT}(P)}{\mathbb{E}[\mathcal{A}(P)]}, \frac{\mathbb{E}[\mathcal{A}(P)]}{\mathrm{OPT}(P)}\right)$$

7.3.1 Derandomization

Given a randomized α-approximation, we may derandomize it by following the method of *conditional expectations*: for any "stage" of the problem in-

duced by a choice, we choose the the next option according to whichever maximizes the conditional expectation of the resulting target value. Aggregated over all of the stages, such a procedure will result in a deterministic algorithm with the same approximation ratio as the randomized algorithm.

7.4 Sublinear Optimization

Often, when subject to performance constraints, we may not even be allotted the resources to read the entire input in the first place (doing so would take $O(n)$ time, at the very least). Instead, we can approximate the answer to a given instance of an optimization problem by scanning some sublinear fraction of the input, and inferring the answer from what we have been able to see.

7.5 Property Testing

Property testing deals with the notion of approximating answers to *decision problems*, rather than optimization problems (it does not make much sense to give an approximate YES/NO answer otherwise!).

A good property testing approximation will output YES if the input is a YES instance, and will output NO if the input is "ϵ-far" from being a YES instance. Inputs that are ϵ-close to being a YES instance may be assigned either outcome.

Property testing is useful when we have access to the problem instance at hand in a *query* model: for example, while we may not be able to process a full graph at once, we can query a graph oracle that tells us if two vertices are connected by an edge. Depending on our precision parameter ϵ, we may run some sublinear number of queries, the conclusions from which we may determine the presence of the property.

8 Streaming Algorithms

Streaming algorithms are presented with inputs similar to an online setting — small parts of an input are presented, one at a time, to our algorithm. In this setting, we are interested in *space minimization* — the goal is to minimize the amount of information stored at any given time, while also maintaining the correctness of a procedure.

8.1 Sketching

Suppose we want to compute some function f on an input X. We can define the *sketch* with respect to f, $C(X)$, which is a compression of X such that it can be stored with limited space, may be updated efficiently when presented with new parts of the input, and can be used to infer the value of $f(X)$.

8.1.1 Min-Hashing

We can utilize properties of random hash functions in order to store information about a set without needing to directly consider members of the set. For example, if we suppose that we have a strongly 2-universal hash function that maps each number uniformly to the interval $[0, 1]$, knowing the minimum value of the hash function over the entire stream can tell us something about the number of distinct elements in the stream, due to order statistics on a uniform distribution. If there are d distinct elemnts getting hashed, then the expected value of the minimum hash is $1/(d+1)$. Thus, if the smallest hash value occurs at value k, then the expected size of the set is around $1/k - 1$.

8.1.2 Majority Counters

Suppose we want to find an element in a set that occurs more than half of the time. To solve this, we keep a counter storing *relative counts*.

> Initialize $C_1(X) = 0$ and $C_2(X) = \text{NULL}$.
> For every element x_i in the input stream:
>
> 1. If x_i is equal to $C_2(X)$, increment $C_1(X)$.
> 2. If $C_2(X)$ is NULL, set $C_2(X) \leftarrow x_i$ and $C_1(X) \leftarrow 1$.
> 3. If x_i is not equal to $C_2(X)$ otherwise, decrement $C_1(X)$.
>
> Return $C_2(X)$ as the element occurring more than half of the time in the stream.

This can further be generalized to finding the k elements that occur more than n/k times in the stream by keeping k separate counters.

8.2 Similarity Search

The Jacquard similarity of two sets X and Y is defined as

$$J(X, Y) = \frac{|X \cap Y|}{|X \cup Y|}$$

Given a collection of n sets $\mathcal{A}_1, \cdots, \mathcal{A}_n$, each of size at most d, and two parameters $s > s' \in [0, 1]$ we would like to efficiently do the following upon a query set $\mathcal{A}$:

1. Output YES if there exists a set $\mathcal{A}_i$ such that $J(\mathcal{A}, \mathcal{A}_i) \geq s$
2. Output NO if, for all sets $\mathcal{A}_i$ in the collection, $J(\mathcal{A}, \mathcal{A}_i) < s'$

For each set, we can store a signature $\sigma_h(\mathcal{A}) = \min_{a \in \mathcal{A}}\{h(a)\}$. Consider, for sets X and Y, when $\sigma_h(X) = \sigma_h(Y)$. As per the min-hash argument, this will occur when the minimum values of X and Y are the same — that is, $|X \cap Y|$ contains the same minimum value as $|X \cup Y|$. Thus,

$$\Pr[\sigma_h(\mathcal{A}) = \sigma_h(\mathcal{A}_i)] = \frac{|\mathcal{A} \cap \mathcal{A}_i|}{|\mathcal{A} \cup \mathcal{A}_i|} = J(\mathcal{A}, \mathcal{A}_i)$$

We can amplify the results from this method by using L independent hash functions, and computing this similarity for each. This produces an *unbiased estimator* for the Jaccard similarities of the sets.